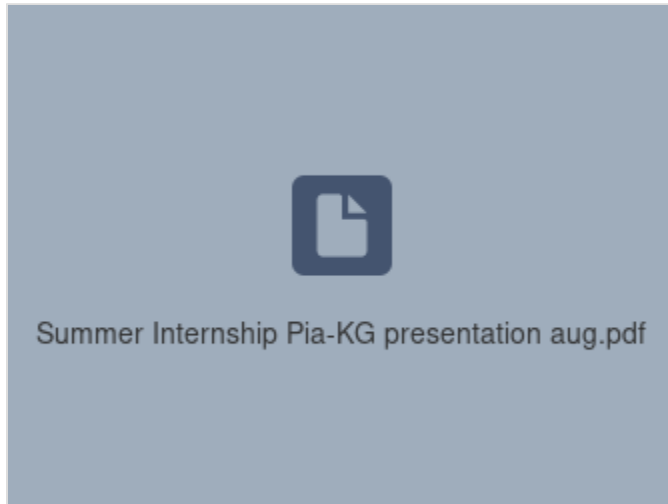


Internship Goals and Plan

See:

- [Current Progress](#)
- [Performance of vector search in UnitDB](#)
- [Tests with Multi-Agentic RAG Graph Prototype](#)

Slides:



Background

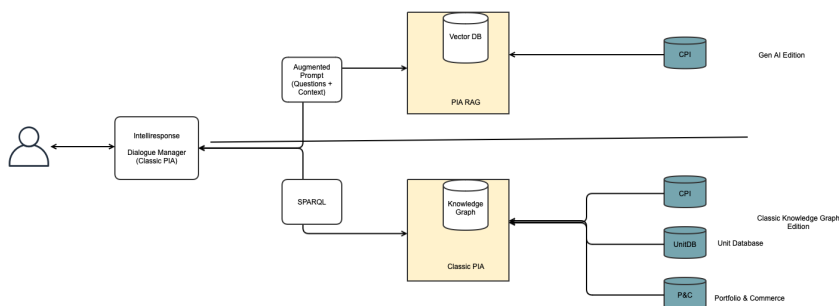
Added from ([LLM & KG Integration](#))

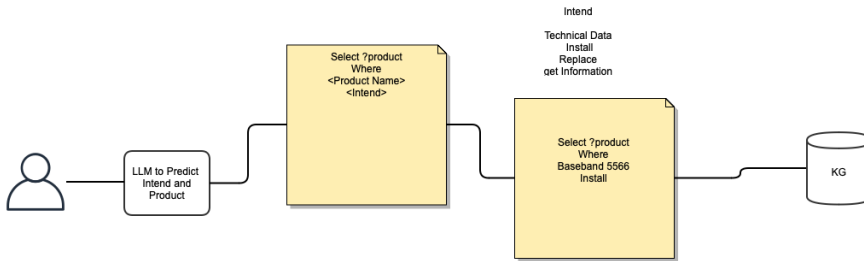
1. LLM prompt refinement.
2. Focus on schema structure helps to enhance the SPARQL Generation performance.
3. LLM may generate a SPARQL query with missing prefixes, syntactic or other errors. In such cases, try to amend this by prompting the LLM to correct the query a certain number of times. In addition to the invalid query, we also provide the LLM with the error message and the ontology schema.
4. Validate the SPARQL query using online SPARQL validators.

Goal:

How to integrate PIA knowledge graphs with RAG solution?

- ☐ PIA RAG supports UnitDB data from Classic Knowledge Graph
- ☐ UnitDB database example questions([Fredrik Smidfelt](#))





Team Vacation Plan

Name	Date
Sarbashis	1st July – 14th July
Saranya	
Dirk	1st July – 14th July & 7th August - 12th August
David	
Aditya	25th - 26th July
Oscar	26th July- 2nd August

See: [Weekly Log](#)

Meeting (20 Jun 2024)

- ☐ Multiple Potential approaches can be applied in RAG in LLMs
 - ☐ The graph-structure could be a guideline to specify how the model should base its decisions
 - ☐ Reveal nearby relevant information and linkings that also help in answering the user's question
 - ☐ Broaden the context of the LLM
 - ☐ Classifier (ML) based product Classification

Activities Plan

- ☒ Execute the code shared by [Saranya Govindaraj](#)
- ☒ Try Prompt Refinement
- ☐ Shared UnitDb SPARQL Template

Tools Needed: (Support needed from Saranya, Dirk, David)

Ericsson OpenAI Package: <https://gitlab.internal.ericsson.com/product-information-assistant/eva/regenx-backend/pia-azure-gateway>

- Use Scoop and/or Homebrew on WSL to install, e.g., OpenJDK
 - <https://brew.sh/> (WSL)
 - <https://scoop.sh/> (Powershell)
- [Protégé](#) = Ontology Editor
- [Command-line and other tools for Jena developers](#)
- [Apache Jena Fuseki](#)= Triple Store (there is also a Docker container with the correct configuration on Git!)
- MinIO
 - Username: pia-user
 - Url: <https://s3-console-eva-dm.azweulx0075.we.az.ericsson.se/>
 - Latest Knowledge Graph: [sandbox / cliopatria-output / 2024-05-31: 2024-05-31_inferred.tar.gz](#)
 - Apply `tar xvf` on archive to get all graphs in Turtle format (the smaller files can be opened in [Protégé](#); UnitDB is too large!)

Additional Data source: PCTD (suggested by @jan-willem)

27 Jun 2024

1. Blank Node Definition:

- a. <https://www.oxfordsemantic.tech/faqs/what-is-a-blank-node>
- b. <https://terminusdb.com/blog/blank-nodes-in-rdf/>
- c. <https://www.w3.org/TR/turtle/>
2. Tried to generate SPARQL Query using LLM
3. Vector based Memory
4. Suggestion by Dirk to look in to the Language tag in the ttl file

03 Jul 2024

Observations:

- Data attributes can sometimes be stored in textual fields, rather than attributes, which can make it difficult for the model to retrieve sufficient context to answer a user question.
 - When using Regular Expression-based search to get nodes, nodes with "@en" and without, could be retrieved. However, language-wise filtering is also doable.
 - JSONifying the schema was feasible, and the node types, object properties, and data properties (3 parts of the schema) were separated, and provided to the LLM, making the separation clear.
- This, however, did not improve the QA capability of the LLM.

```

Include only necessary prefixes and don't use any undefined prefixes, based on the below schema.

These are the available node types:
{schema_nodes}

These are the object properties, i.e., relationships between objects:
{schema_objects}

These are the data properties, i.e., relationships between objects and literals:
{schema_data}

Note: Be as concise as possible.

```

- Optional clauses can be added in the generated SPARQL query, to make the LLM look in the text-based attributes, for if more contextual information can be found, to answer a question.

Error rendering macro 'jira'

java.io.IOException: Error on line 1: The markup in the document preceding the root element must be well-formed.

Optical Indicator blank nodes were found, however they related to radio_access-technology/<lte, gsm, wcdma>, and are all currently present optical indicators. It is, however, not traceable to a baseband model, which makes the querying challenging. Hence, getting optical indicator information if present in text fields, is added as an OPTIONAL statement in the SPARQL query to cover both data sources.

SPARQL-validation and iterative improvement:

- First version complete, implemented with SPARQL-wrapper. Gives error feedback, which is feed back into the llm, which can fix syntax errors.
- We need to test how well this works and how many iterations should be run

Jsonify Schema:

- Done, but a small bug related to the implementation. Will be fixed if we think the schema restructuring works as a concept

04 Jul 2024 weekly meeting:

- More testing on SPARQL-validation performance and functionality needs to be done. Refinement of re-generation prompt template is needed.
- Look if graph-RAG has Langchain support. (low-priority).
- Try to find more example questions, to make testing easier.

09 Jul 2024 Update meeting:

- Schema restructure done, by creating CustomRdfGraph class, and changing how the schema is generated. Still some problem with hidden properties and relations, not picket up by the schema, which should be explored more in the future.
- Discussed implementing memory to the qa_chain, for better conversation.
- General discussion about how to continue with the project. Added new confluence page with the current progress and TODOs, where we have the general work progress and plans.

12 Jul 2024

- Memory component was added in the sub-chain, and passed to the summary generator / answering chain (in the form of ConversationBufferMemory) - [Commit](#)

- KnowledgeGraphSparQLQChain consists of 3 sub-chains - an **Intent Chain**, a **SparQL Generation Chain** (creating a sparql query), and the **QA chain** (generating a suitable response in natural text).
 - Optionally, in conjunction with the Generation Chain, if a generated SparQL query is invalid, or has syntax issues, a new **SparQL Regeneration Chain** is invoked, that:
 - would prompt the language model with the previous false SparQL query, and the error it received (e.g. "invalid prefix"), and request it to fix the SparQL query a maximum of 3 times, before returning a response "No data could be found to answer the question", instead of crashing.
- Experimented with new questions to see whether memory is retained. The model remembered the response from the very first question, that the product name is Baseband 6303, and used that in the second and third question.

```
"What is the product name for the product number 'KDU137974/11'?", # Answer: Baseband 6303
"How do I power it up after installation?", # it = Baseband 6303 (successfully identified by the chain)
"What is its product number?" # it = Baseband 6303 (also identified here)
```

- Memory contents:

```
[HumanMessage(content="What is the product name for the product number 'KDU137974/11'?" ),
 AIMessage(content="The product name for the product number 'KDU137974/11' is 'Baseband 6303'."),
 HumanMessage(content='How do I power it up after installation?'),
 AIMessage(content='To power up the Baseband after installation, you should first make sure that the
 Baseband is grounded according to the instructions. Then, switch on the external power to the Baseband.
 After that, you can check the Baseband by opening the cover for the optical indicators and ensuring that
 the operation optical indicator is green. If any red optical indicator is on, you should contact the OMC
 for maintenance support. '),
 HumanMessage(content='What is its product number?'),
 AIMessage(content="The product number for 'Baseband 6303' is 'KDU137974/11'."),
 HumanMessage(content='Who should contact for maintenance support, if a red optical indicator is on?'),
 AIMessage(content='If a red optical indicator is on, you should contact the OMC for maintenance
 support. ')]
```

Next Steps:

- ☒ Try with a constricted window of memory, to keep the prompt under the token limit. (ConversationBufferWindowMemory)
- ☒ SPARQL validation using LLM and SPARQLWrapper [Oscar Stackenland](#)
- ☒ Testing SPARQL validation
- ☒ Explicit Schemas can be useful
- ☒ Jsonify the schema
- ☒ Vector embedding the entities and information from the knowledge Graph ([GraphRAG](#)) [Aditya Khadikar](#)
UNITDB to embeddings: [Tests with Embedding KG Content](#)

Meeting 16 Jul 2024

- ☐ SPARQL Query Generation using LLM
- ☐ Generating Knowledge Graph using LLM. Ref: https://python.langchain.com/v0.2/docs/how_to/graph_constructing/#llm-graph-transformer